

Data Structure(CSL202)

Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)

Operations in BST



Minimum in BST

```
minimum(Node n)

// Traverse left while possible

1. while n ≠ NULL and left(n) ≠ NULL do

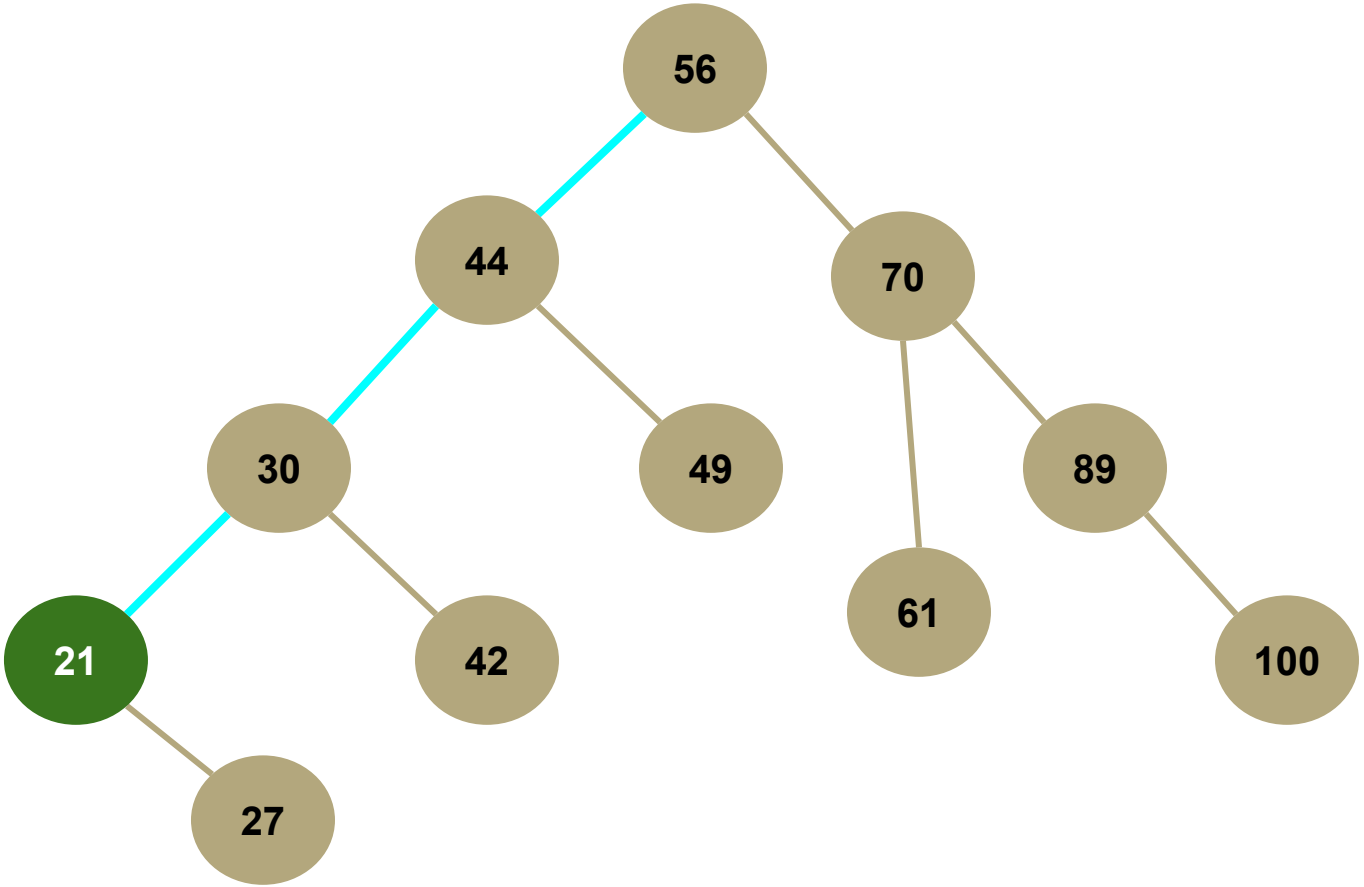
// Move to the left child

2.      n := left(n)

// Return the leftmost (minimum) node

3. return n
```

Minimum in BST



Minimum in BST Complexity

Best/Average Case:

- If the BST is balanced, height $h \approx \log n$.
- So, running time = $O(\log n)$.

Worst Case:

- If the BST is skewed (like a linked list), height $h = n$.
- So, running time = $O(n)$.

Operations in BST

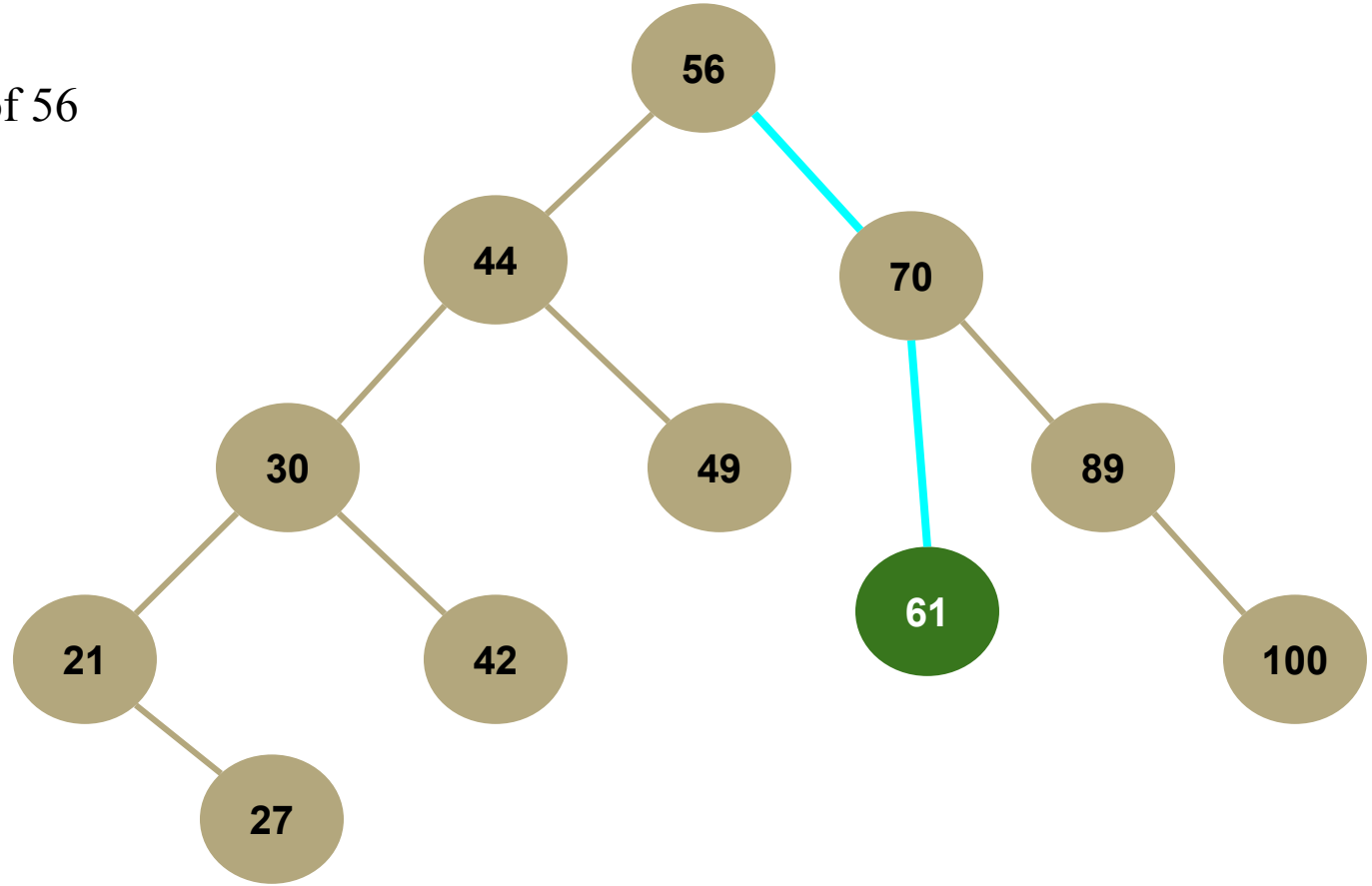


Successor in BST

- The successor in a BST is the next node in the inOrder traversal of BST
- It is the smallest node value that is greater than the given node value

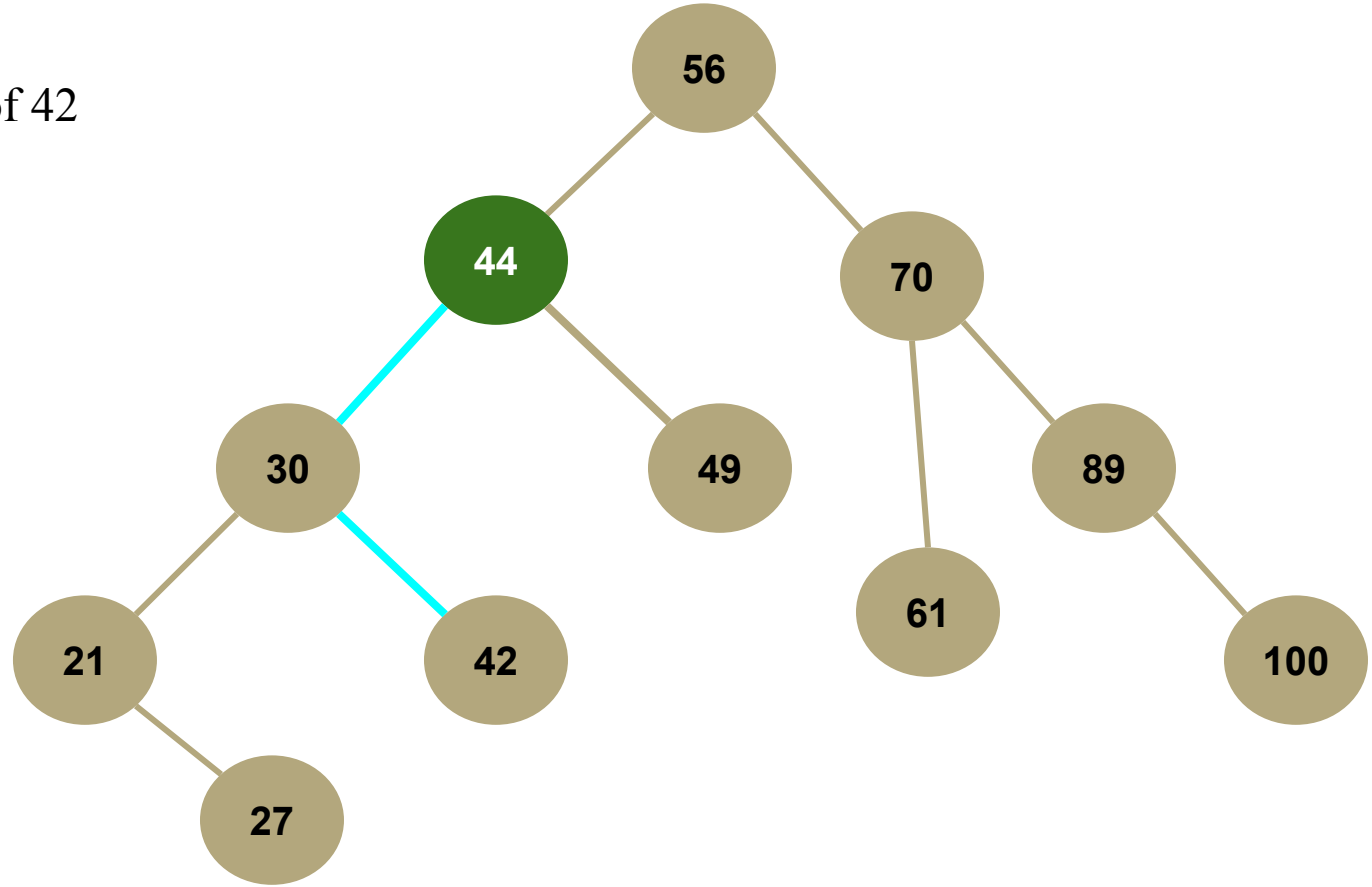
Successor in BST

Successor of 56



Successor in BST

Successor of 42



Successor in BST

```
successor(BST T, Node n)
1. if right(n) ≠ NULL then
2.     return minimum(right(n))
       // Successor is the smallest node in the right subtree
3. while parent(n) ≠ NULL and right(parent(n)) = n do
4.     n := parent(n)
       // Move up until we come from the left child
5. return parent(n)
```

Successor in BST Complexity

Best/Average Case:

- If the BST is balanced, height $h \approx \log n$.
- So, running time = $O(\log n)$.

Worst Case:

- If the BST is skewed (like a linked list), height $h = n$.
- So, running time = $O(n)$.

Operations in BST



Deletion from BST

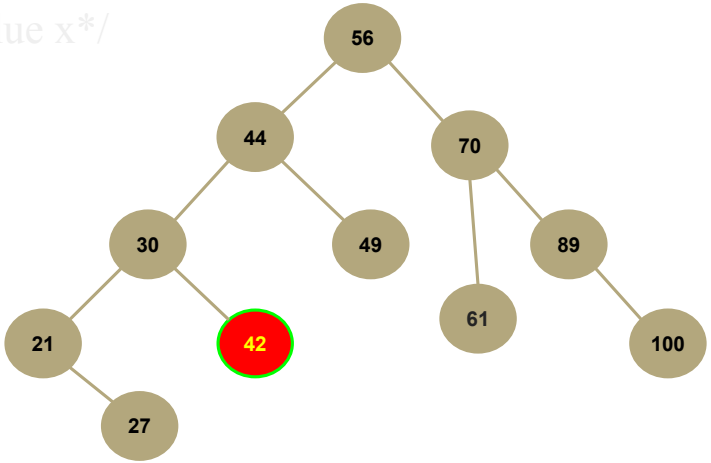
Only the value to be deleted is given:

- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n

Deletion from BST

Only the value to be deleted is given:

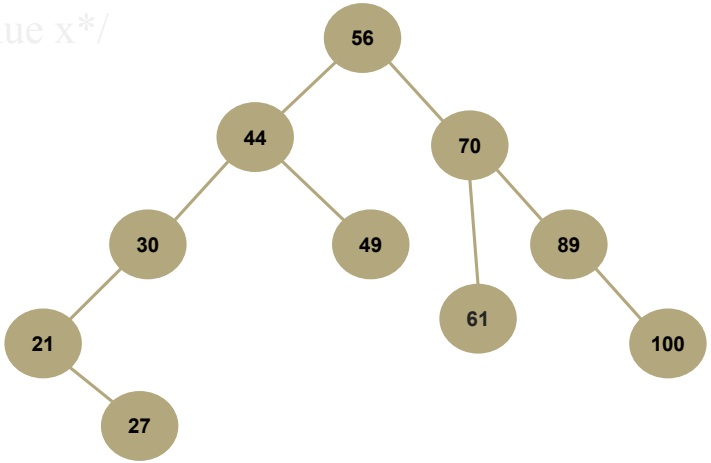
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

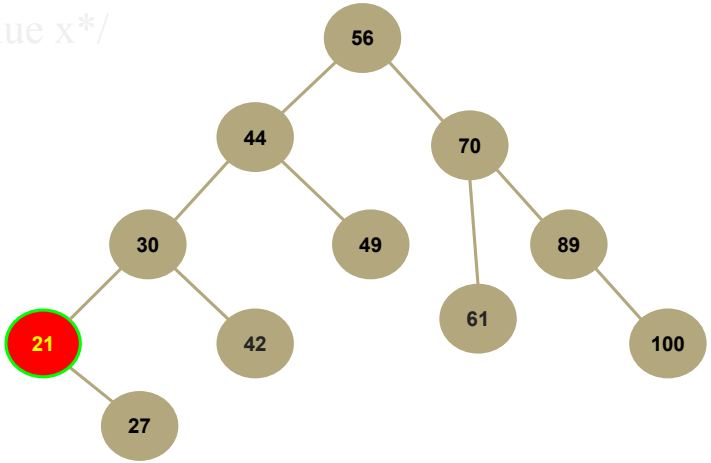
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

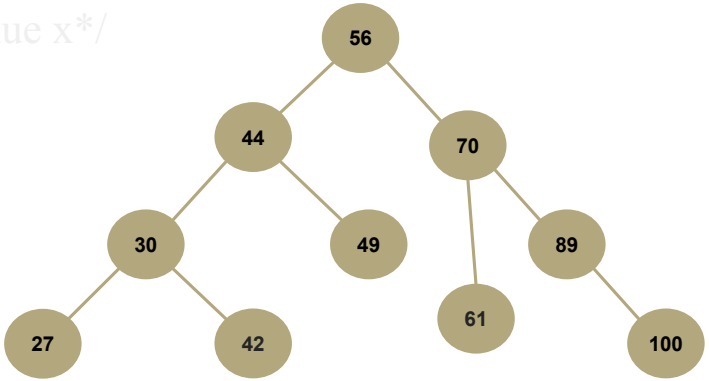
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

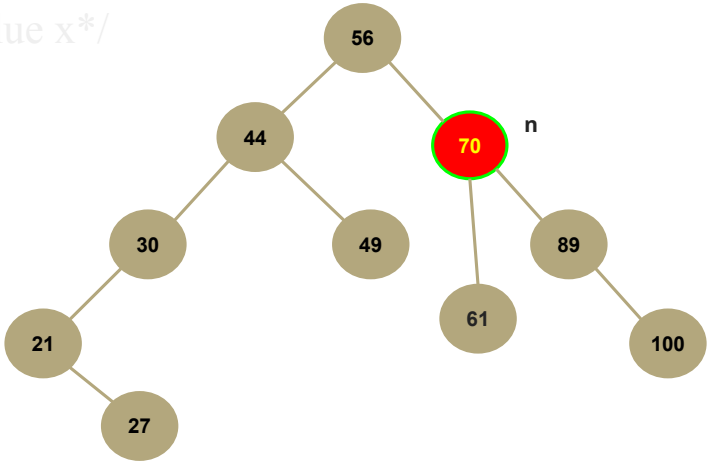
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

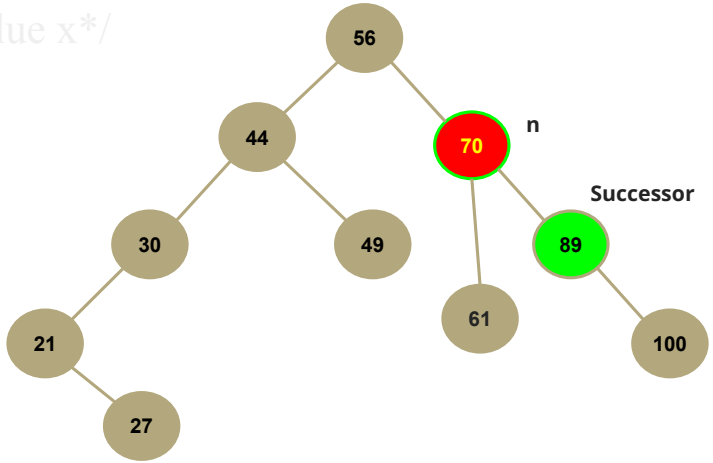
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

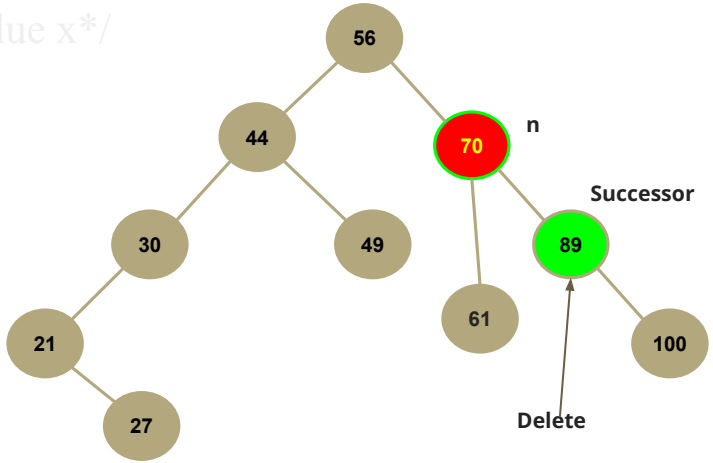
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

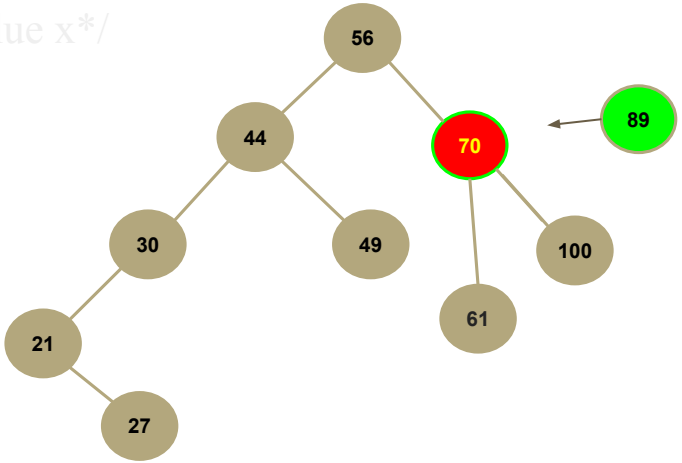
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

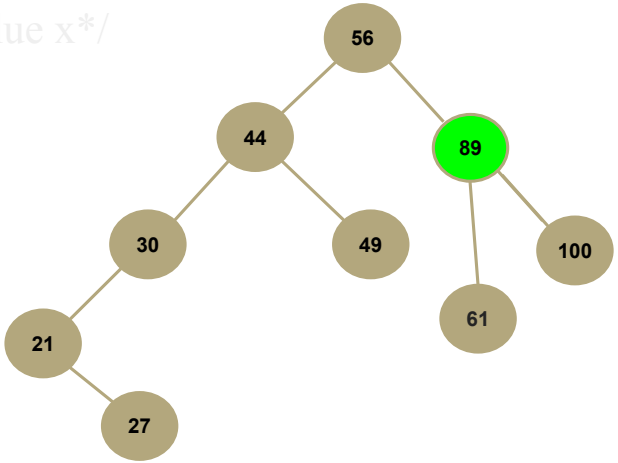
- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

- Step 1: Find the node corresponding to the value x
  - Search(BST T, int k) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n



Deletion from BST

Only the value to be deleted is given:

- Step 1: Find the node corresponding to the value x
  - Find(BST T, int x) /*it returns the node n with value x*/
- Step 2: Given node n there are three cases:
  - Case1: n has no children
    - Replace node n with null pointer
  - Case 2: n has only one child (left or right)
    - Replace node n with its only child
  - Case 3: n has two children
    - Delete Successor (BST T, Node n) and replace it with node n

Deletion from BST

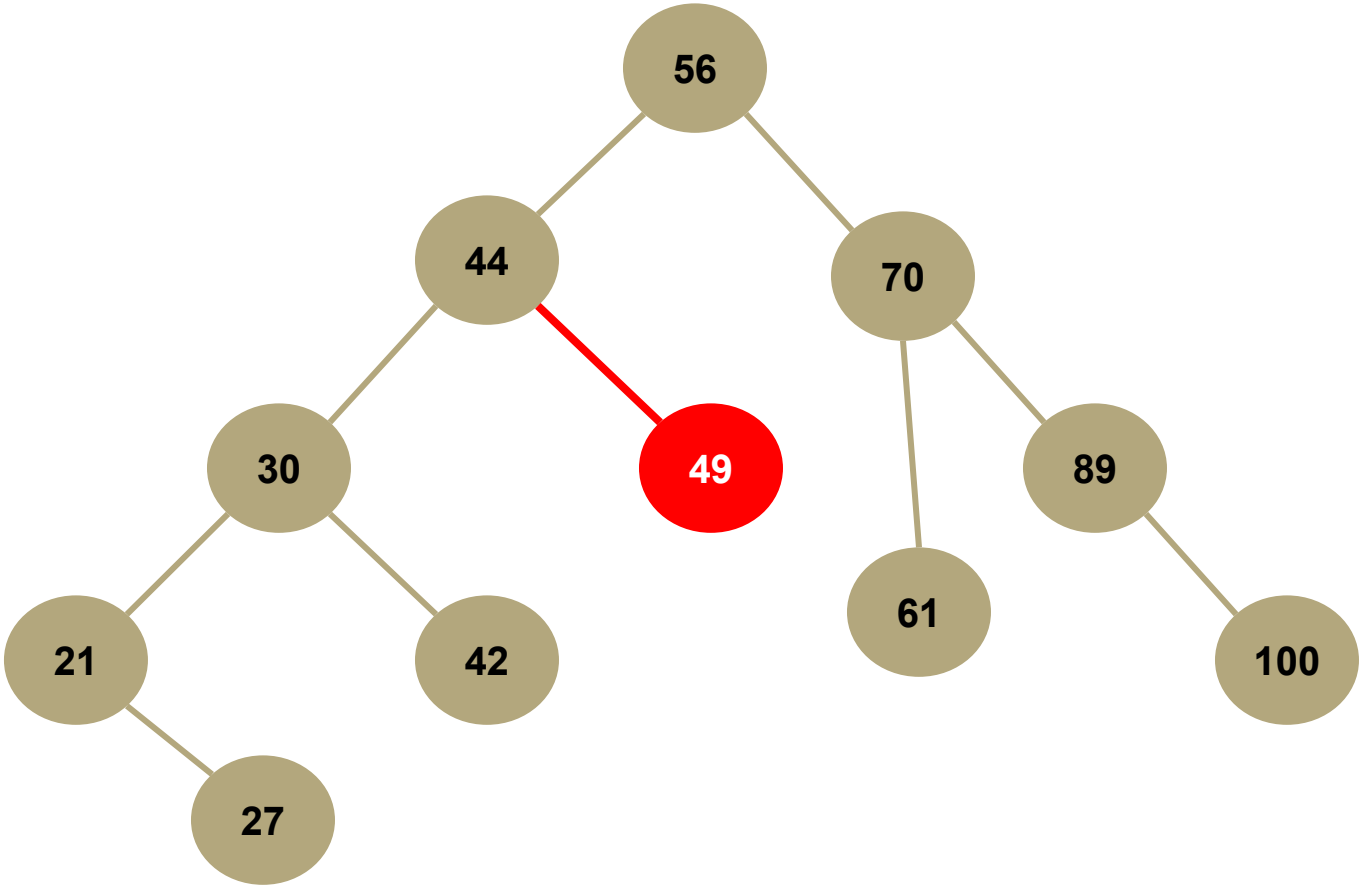
```
delete(BST T, Node n) // Directly the node n is given which has to be deleter
1. if (left(n) = NULL && right(n) = NULL)                    // Step 2: Case 1
2.
 if y ≠ n then
3.     key(n) := key(y)                                      // Copy y's data into n
4. x := (left(y) = NULL) ? right(y) : left(y)                // x is y's only child (or NULL if y is a leaf)
5. if x ≠ NULL then
6.     parent(x) := parent(y)                                //y is not a leaf, update the parent of x
7. if parent(y) = NULL then
8.     root(T) := x                                          // y was the root, therefore x is root now
9. else if left(parent(y)) = y then
10.    left(parent(y)) := x                                  //Remove y from the tree
11. else
12.    right(parent(y)) := x                                 //Remove y from the tree
```

Deletion from BST

```
delete(BST T, Node n) // Directly the node n is given which has to be delete
1. y := (left(n) = NULL or right(n) = NULL) ? n : Successor(T, n)        // y will be deleted
2. if y ≠ n then
3.     key(n) := key(y)                                                  // Copy y's data into n
4. x := (left(y) = NULL) ? right(y) : left(y)                            // x is y's only child (or NULL if y is a leaf)
5. if x ≠ NULL then
6.     parent(x) := parent(y)                                            //y is not a leaf, update the parent of x
7. if parent(y) = NULL then
8.     root(T) := x                                                      // y was the root, therefore x is root now
9. else if left(parent(y)) = y then
10.    left(parent(y)) := x                                              //Remove y from the tree
11. else
12.    right(parent(y)) := x                                             //Remove y from the tree
```

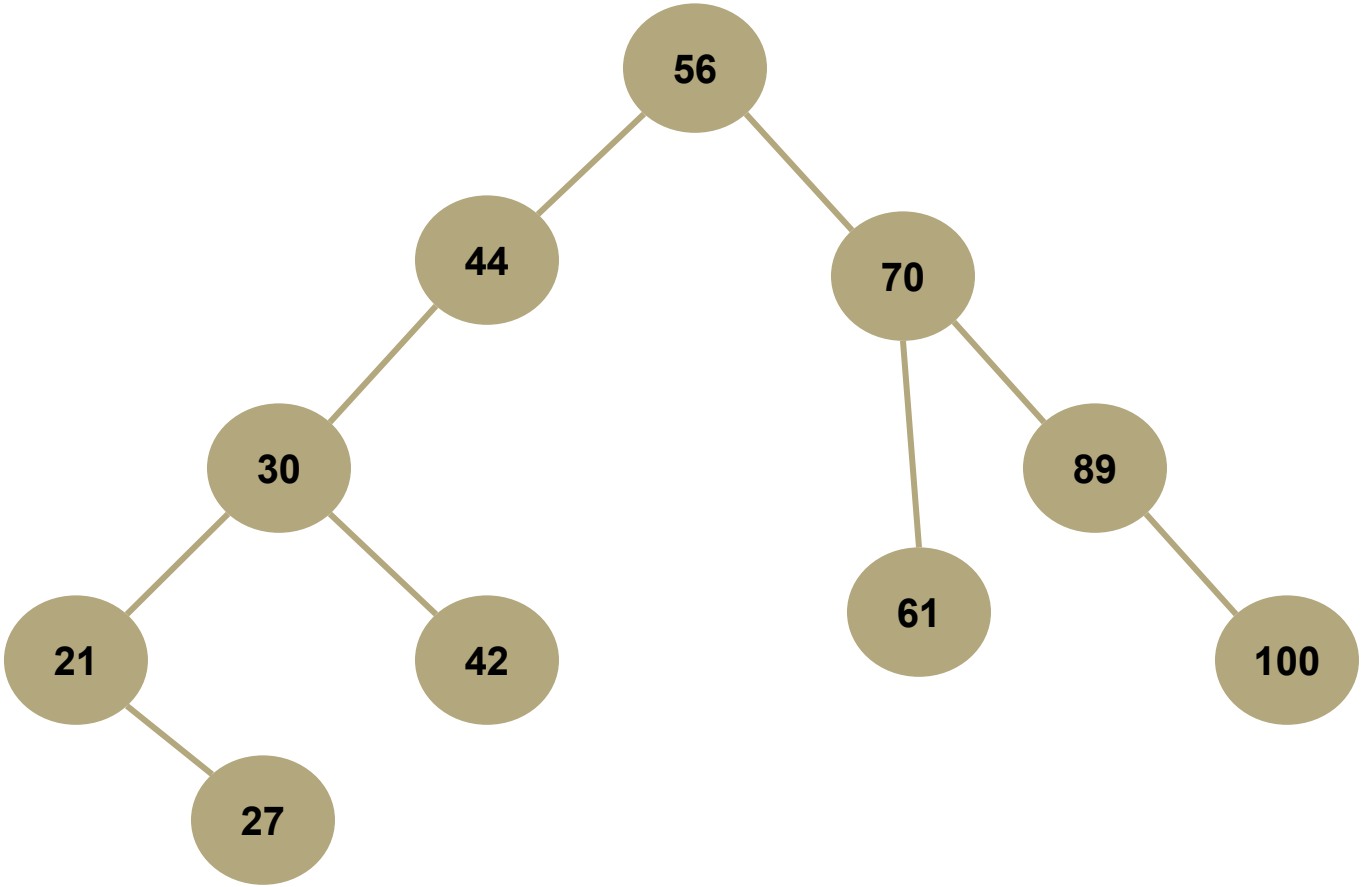
Deleting a leaf

Delete 49



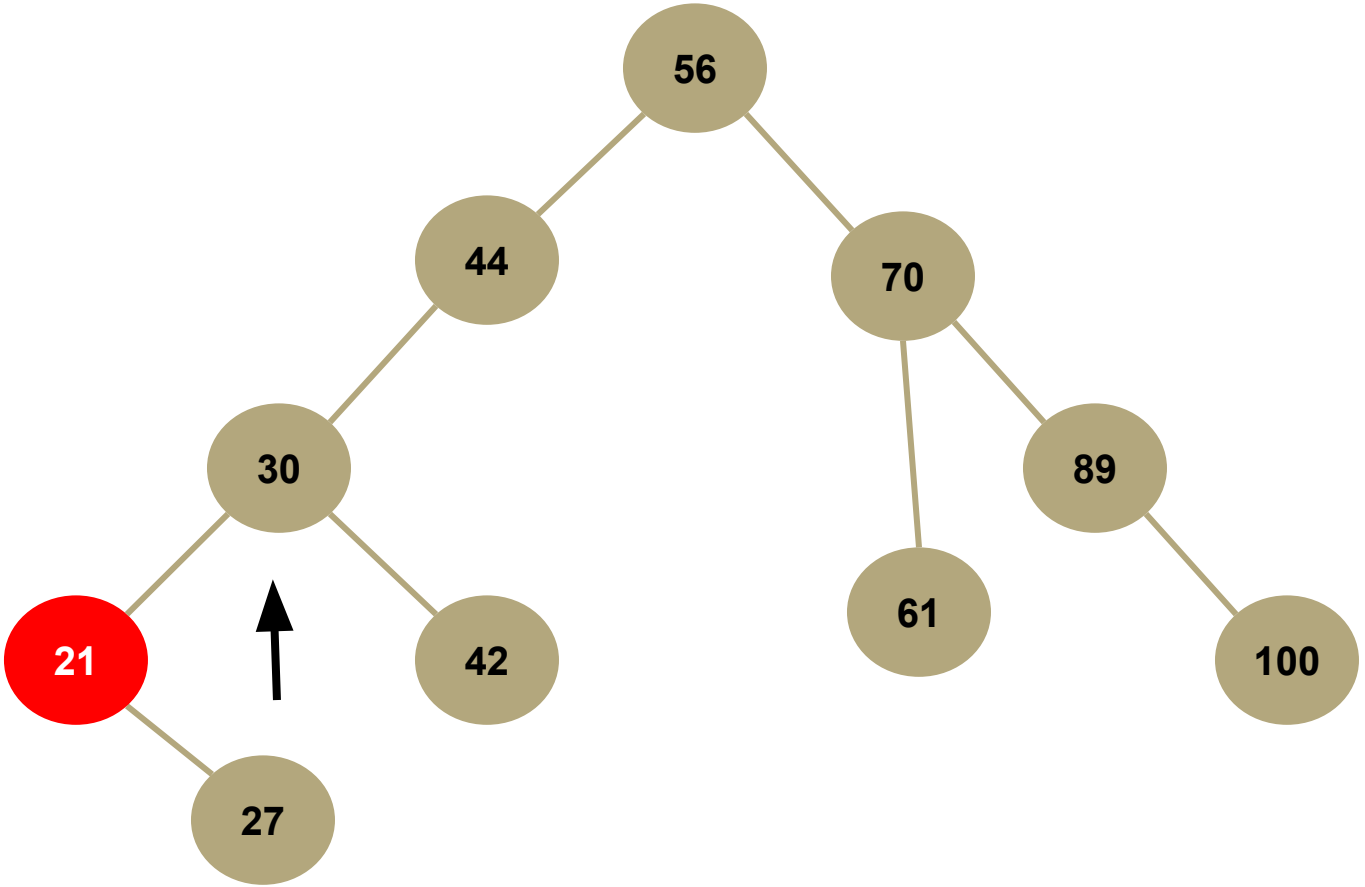
Deleting a leaf

Delete 49



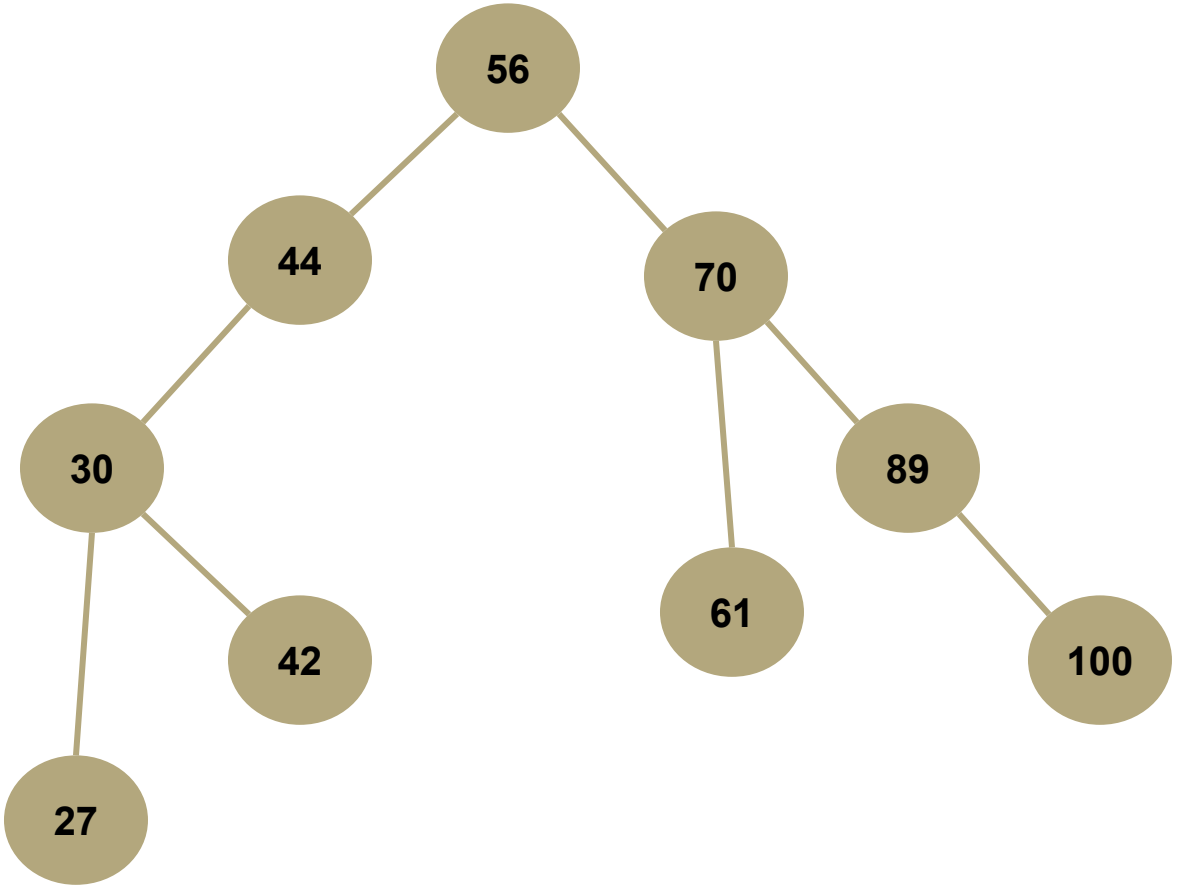
Deleting a Node with a Single Child

Delete 21



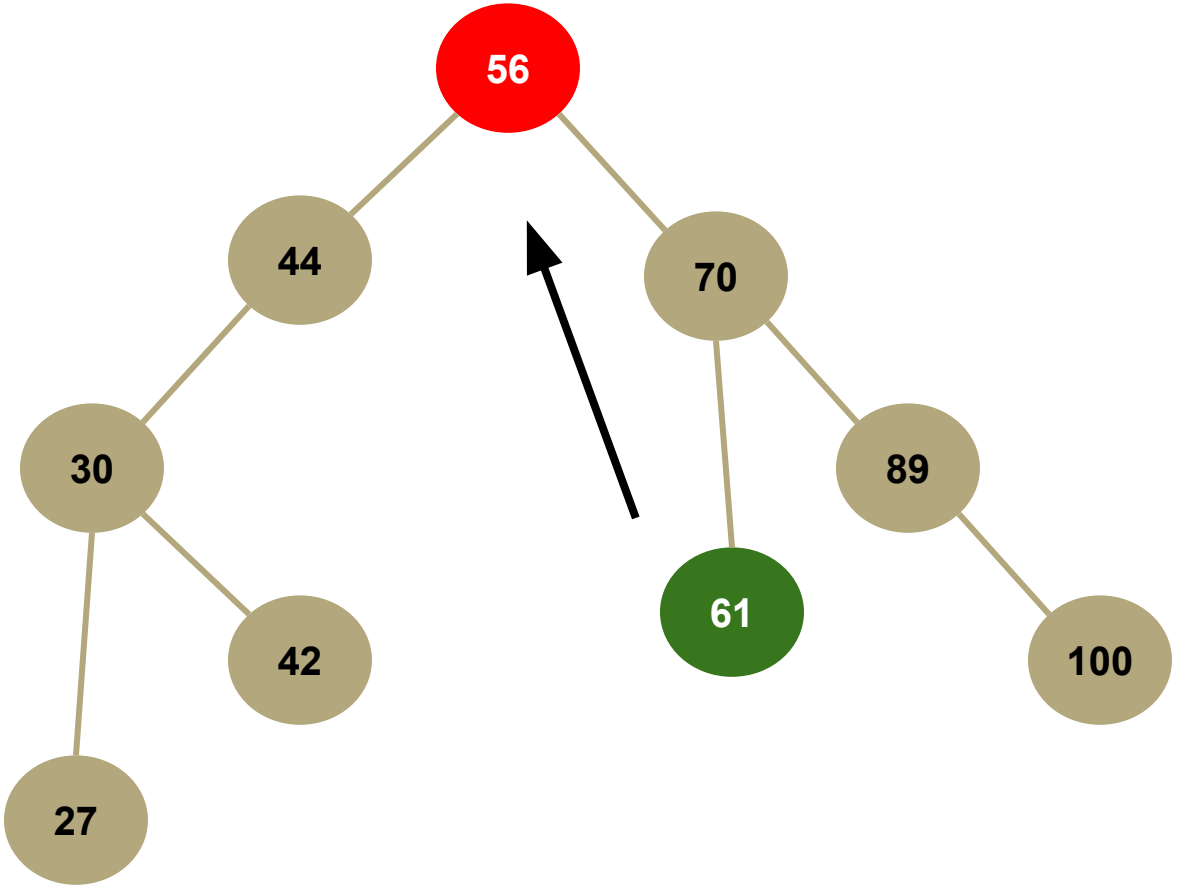
Deleting a Node with a Single Child

Delete 21



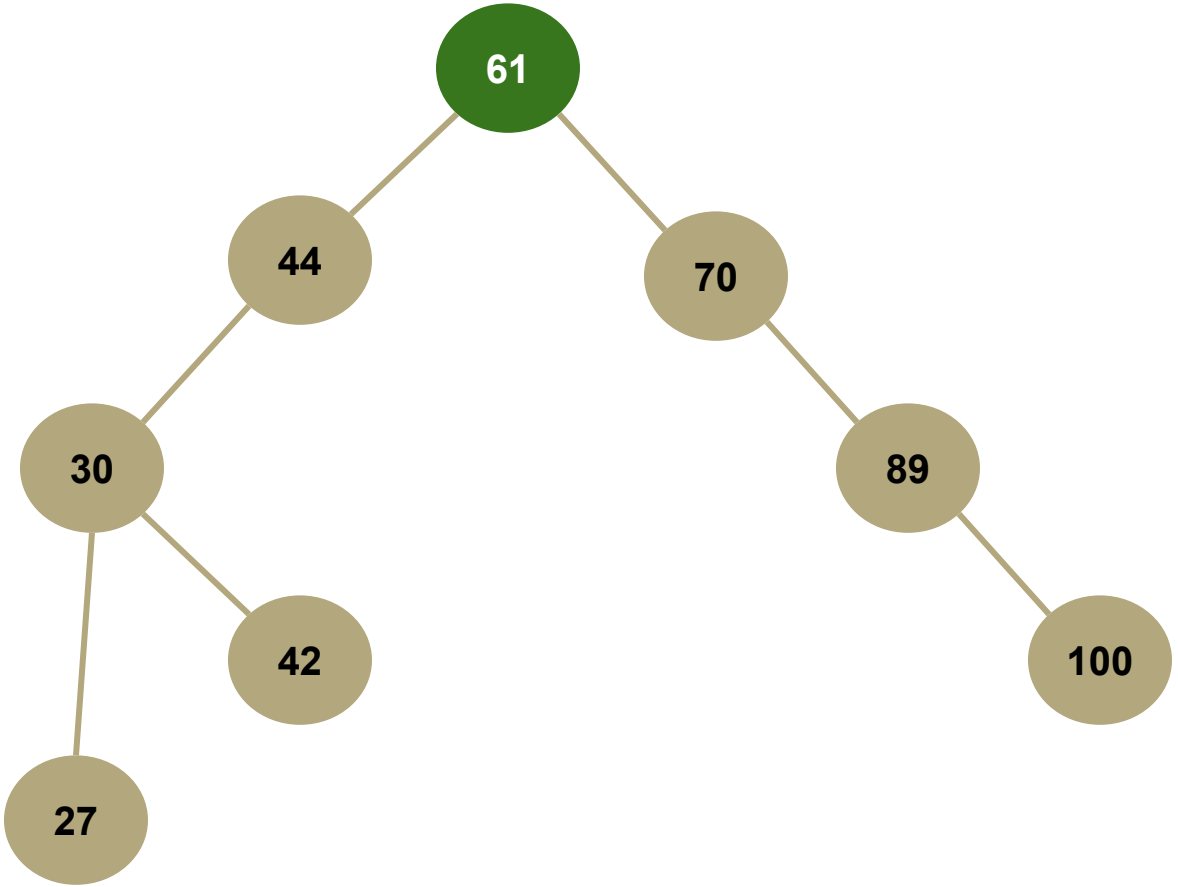
Deleting a Node with Both Child

Delete 56



Deleting a Node with Both Child

Delete 56



Deletion from BST Complexity

Best/Average Case:

- If the BST is balanced, height h ≈ log n.
- So, running time = O(log n).

Worst Case:

- If the BST is skewed (like a linked list), height h = n.
- So, running time = O(n).